UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

CS2042 - BASES DE DATOS II

> Reinventa el mundo <

Organización Física y Almacenamiento

SEMANA 02

Heider Sanchez
hsanchez@utec.edu.pe

TRANSFORMATEC

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

> Reinventa el mundo <

3.

Métodos de Organización de Archivos

Sequential File

File Structures: An object-oriented approach with C++, Michael J. Folk. Addison Wesley, 3rd Edition, 1998.

Sequential [Ordered] File

- Los registros se guardan de manera secuencial, usualmente basándose en la clave primaria de la tabla.
- Esto facilita un acceso ordenado y eficiente a los registros.
- Para lograr esto, es fundamental que el archivo conserve los registros organizados físicamente de acuerdo con el valor del campo de búsqueda (clave).
- Se puede acceder a un registro con O(log n) usando la estrategia de búsqueda binaria.

datos.dat

Cod	Nombre	Ciclo
P-102	Keiko	5
P-250	Rafael	7
P-231	Carlos	3
P-452	Jorge	5
P-587	Marisol	1
P-612	Ricardo	3
P-682	Fernando	9

Sequential File: Operaciones

Búsqueda binaria en el archivo:

- El algoritmo de búsqueda binaria se utiliza para encontrar un registro en el archivo con un valor de búsqueda *k.*
- Se necesitan **O(log N)** accesos a la memoria secundaria.
- Durante la búsqueda, **es importante ignorar los registros que están marcados como eliminados.**

```
Require: k
  l = 0
  u = size() − 1
  while u >= l do
      m = ⌊(l + u)/2⌋
      re = readRecord(m)
      if re.key < k then
          u = m − 1
      else if re.key > k then
          l = m + 1
      else
          return m
      end if
  end while
```

Sequential File: Operaciones

¿Cómo se puede mantener el archivo siempre ordenado?

¿Cuál es el costo de insertar?

¿Cuál es el costo de eliminar?

datos.dat

Cod	Nombre	Ciclo
P-102	Keiko	5
P-250	Rafael	7
P-231	Carlos	3
P-452	Jorge	5
P-587	Marisol	1
P-612	Ricardo	3
P-682	Fernando	9

Sequential File: Operaciones

Estrategia del espacio extra (1):

- Las inserciones más recientes se guardan en un espacio extra.
- Debe haber un límite máximo de K registros en ese espacio adicional.
- La búsqueda debe realizarse en ambos espacios.
- De forma periódica, el archivo de datos debe ser reconstruido utilizando los registros del espacio extra.

datos.dat

Cod	Nombre	Ciclo
P-102	Keiko	5
P-250	Rafael	7
P-231	Carlos	3
P-452	Jorge	5
P-587	Marisol	1
P-612	Ricardo	3
P-682	Fernando	9

aux.dat

P-312	Gabriel	3
P-123	Diana	6

...

Sequential File: Operaciones

Estrategia del espacio adicional (2):

- En el espacio adicional, también es posible organizar los registros según la clave.
- En este caso,
  - ¿Cuál sería la complejidad de acceso a memoria secundaria en operaciones de búsqueda e inserción?
  - ¿Cuál sería la complejidad al realizar la reconstrucción del archivo de datos?

datos.dat

Cod	Nombre	Ciclo
P-087	Saulo	5
P-150	Carlos	7
P-262	Mabel	3
P-331	Josimar	5
P-487	Jorge	1
P-512	Cinthya	3
P-682	Andrea	9

aux.dat

P-223	Gabriel	6
P-312	Diana	3
P-655	María	7

...

Sequential File: Operaciones

Estrategia de registros enlazados:

- Determinar el lugar donde se incorporará el nuevo registro.
  - Si el espacio está disponible, proceder a la inserción.
  - Si no, guardar el registro en el espacio auxiliar.
    - **En este caso, será necesario actualizar los punteros.**
- Es fundamental reorganizar el archivo de datos de forma regular, integrando los registros del espacio auxiliar de manera ordenada.

datos.dat

Id	Nombre	Ciclo	
P-102	Andrea	5	
P-250	Carlos	7	
P-362	Cinthya	3	
P-231	Josimar	5	
P-087	Jorge	1	
P-312	Mabel	3	
P-982	Saulo	9	

aux.dat

P-312	Gabriel	3	
P-123	Diana	6	

...

datos.dat

	Id	Nombre	Ciclo	1(d)
1	P-102	Andrea	5	2(d)
2	P-250	Carlos	7	3(d)
3	P-362	Cinthya	3	4(d)
4	P-231	Josimar	5	5 (d)
5	P-087	Jorge	2	6 (d)
6	P-312	Mabel	3	7(d)
7	P-982	Saulo	9	-1 (d)

Insertar

P-088	Gabriel	4	

P-312	Gonzalo	3	

P-087	Maria	2	

P-014	Abel	5	

aux.dat

1
2
3
4

Sequential File: Operaciones

Eliminación de un registro:

- Se emplean los punteros para omitir los registros que han sido eliminados.
- Durante la reconstrucción del archivo, estos serán eliminados por completo.

Id	Nombre	Ciclo	
P-102	Andrea	5	
P-250	Carlos	7	
P-362	Cinthya	3	
~~P-231~~	~~Josimar~~	~~5~~	-1
P-087	Jorge	1	
~~P-312~~	~~Mabel~~	~~3~~	-1
P-982	Saulo	9	

Eliminar Josimar y Mabel

datos.dat

	Id	Nombre	Ciclo	4(a)
1	P-102	Andrea	5	2(d)
2	P-250	Carlos	7	3(d)
3	P-362	Cinthya	3	1 (a)
4	P-231	Josimar	5	5 (d)
5	P-087	Jorge	2	6 (d)
6	P-312	Mabel	3	3(a)
7	P-982	Saulo	9	-1 (d)

Eliminar: Josimar, Jorge

→ Antes de eliminar

aux.dat

1	P-088	Gabriel	4	2(a)
2	P-312	Gonzalo	3	4(d)
3	P-087	Maria	2	7(d)
4	P-014	Abel	5	1(d)

datos.dat

	Id	Nombre	Ciclo	4(a)
1	P-102	Andrea	5	2(d)
2	P-250	Carlos	7	3(d)
3	P-362	Cinthya	3	1(a)
4	~~P-231~~	~~Josimar~~	~~5~~	~~0~~
5	~~P-087~~	~~Jorge~~	~~2~~	~~0~~
6	P-312	Mabel	3	3(a)
7	P-982	Saulo	9	-1 (d)

aux.dat

1	P-088	Gabriel	4	2 (a)
2	P-312	Gonzalo	3	6(d)
3	P-087	Maria	2	7(d)
4	P-014	Abel	5	1(d)

Eliminar: Josimar, Jorge

→ Después de eliminar

datos.dat

	Id	Nombre	Ciclo	1(d)
1	P-102	Andrea	5	2(d)
2	P-250	Carlos	7	3(d)
3	P-362	Cinthya	3	2 (a)
4	~~P-231~~	~~Josimar~~	~~5~~	~~0~~
5	~~P-087~~	~~Jorge~~	~~2~~	~~0~~
6	P-312	Mabel	3	3 (a)
7	P-982	Saulo	9	0 (d)

aux.dat

1	P-088	Gonzalo	4	6 (d)
2	P-312	Gabriel	3	1 (a)
3	P-087	Maria	2	7 (d)

Reconstrucción

Id	Nombre	Ciclo	
P-102	Andrea	5	
P-250	Carlos	7	
P-362	Cinthya	3	
P-312	Gabriel	3	
P-088	Gonzalo	4	
P-312	Mabel	3	
P-087	Maria	2	
P-982	Saulo	9	

Insertar ordenadamente siguiendo los punteros

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

datos.dat

	7	Id	Nombre	Ciclo
ordenado	1	P-102	Andrea	5
	2	P-250	Carlos	7
	3	P-362	Cinthya	3
	4	P-231	Josimar	5
	5	P-087	Jorge	2
	6	P-312	Mabel	3
	7	P-982	Saulo	9
no ordenado	8	P-088	Gonzalo	4
	9	P-312	Gabriel	3
	10	P-087	Maria	2

Reconstrucción

Id	Nombre	Ciclo
P-102	Andrea	5
P-250	Carlos	7
P-362	Cinthya	3
P-312	Gabriel	3
P-088	Gonzalo	4
P-231	Josimar	5
P-087	Jorge	2
P-312	Mabel	3
P-087	Maria	2
P-982	Saulo	9

En la práctica, el espacio auxiliar se maneja en el mismo archivo de datos

datos.dat

	7	Id	Nombre	Ciclo	Next
ordenado	1	P-102	Andrea	5	**2**
	2	P-250	Carlos	7	**3**
	3	P-362	Cinthya	3	**9**
	4	~~P-231~~	~~Josimar~~	~~5~~	~~**0**~~
	~~**5**~~	~~P-087~~	~~Jorge~~	~~2~~	~~**0**~~
	6	P-312	Mabel	3	**10**
	7	P-982	Saulo	9	**-1**
no ordenado	8	P-088	Gonzalo	4	**6**
	9	P-312	Gabriel	3	**8**
	10	P-087	Maria	2	**7**

Reconstrucción

Id	Nombre	Ciclo	
P-102	Andrea	5	**2**
P-250	Carlos	7	**3**
P-362	Cinthya	3	**4**
P-312	Gabriel	3	**5**
P-088	Gonzalo	4	**6**
P-312	Mabel	3	**7**
P-087	Maria	2	**8**
P-982	Saulo	9	**-1**

En la práctica, el espacio auxiliar se maneja en el mismo archivo de datos

Sequential File: Resumen

datos.dat

Id	Nombre	Ciclo
P-102	Andrea	5
P-250	Carlos	7
P-362	Cinthya	3
P-231	Josimar	5
P-087	Jorge	1
P-312	Mabel	3
P-982	Saulo	9

aux.dat

P-312	Gabriel	3
P-123	Diana	6

- Search:
- Insert:
- Rebuild:
- Remove:

Dos formas de manejar el espacio auxiliar

datos.dat

Id	Nombre	Ciclo	
P-102	Andrea	5	
P-250	Carlos	7	
P-362	Cinthya	3	
P-231	Josimar	5	
P-087	Jorge	1	
P-312	Mabel	3	
P-982	Saulo	9	

aux.dat

P-312	Gabriel	3	
P-123	Diana	6	

- Search:
- Insert:
- Rebuild:
- Remove:

Sequential File

- Ventajas
  - Búsquedas eficientes
- Desventajas
  - Son difíciles de mantener.
    - Inserciones y eliminaciones requieren de una localización previa del registro.
  - Costo extra para reorganizar el archivo.

Id	Nombre	Ciclo	
P-102	Andrea	5	
P-250	Carlos	7	
P-362	Cinthya	3	
P-231	Josimar	5	
P-087	Jorge	1	
P-312	Mabel	3	
P-982	Saulo	9	

Binary Search Tree

- ¿Qué tal si organizamos los registros utilizando la estrategia de Árbol Binario de Búsqueda?
  - Lo denominaremos BST File
- ¿De qué manera se modifica el archivo?
- ¿Cuál es la complejidad para insertar, eliminar y buscar?
- ¿Cómo aseguramos que el árbol se mantenga balanceado en todo momento (AVL File)?

Binary Search Tree

	Cod	**Nombre**	**Ciclo**	**Left**	**Right**
1	P-271	Josimar	5	*2*	*3*
2	P-255	Manuel	8	*4*	*-1*
3	P-362	Cinthya	3	*-1*	*5*
4	P-224	Andrea	2	*-1*	*-1*
5	P-887	Benjamin	9	*-1*	*-1*

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

> Reinventa el mundo <

Laboratorio 03

TRANSFORMATEC

Agrupando Registros en Páginas

- En la práctica, el archivo secuencial se organiza en paginas manteniendo el orden lexicográfico en función de la llave.

Cod	Nombre	Ciclo
P-102	Keiko	5
P-250	Rafael	7
P-231	Carlos	3
P-452	Jorge	5
P-587	Marisol	1
P-612	Ricardo	3
P-682	Fernando	9

Page 1

RID	Cod	Nombre	Ciclo
1	P-102	Keiko	5
2	P-250	Rafael	7
3	P-231	Carlos	3

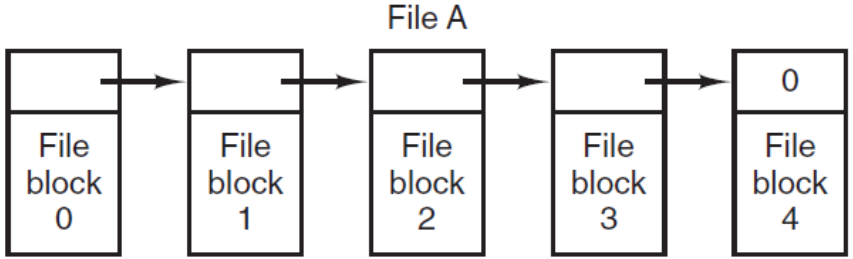
Page 2

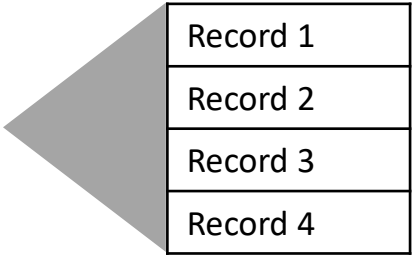
RID	Cod	Nombre	Ciclo
4	P-452	Jorge	5
5	P-587	Marisol	1
6	P-612	Ricardo	3

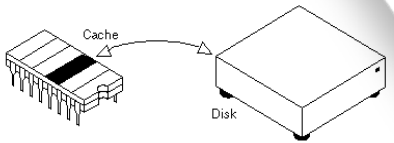
...

Agrupando Registros en Páginas

- **Los bloques** son las unidades básicas de almacenamiento en un dispositivo físico, como un disco duro o una unidad SSD.
  - El sistema de archivos organiza los datos de un archivo en bloques para su almacenamiento y recuperación.
  - El tamaño de los bloques lo define el sistema de archivos y el dispositivo de almacenamiento, **son unidades del tamaño que pueden variar.**

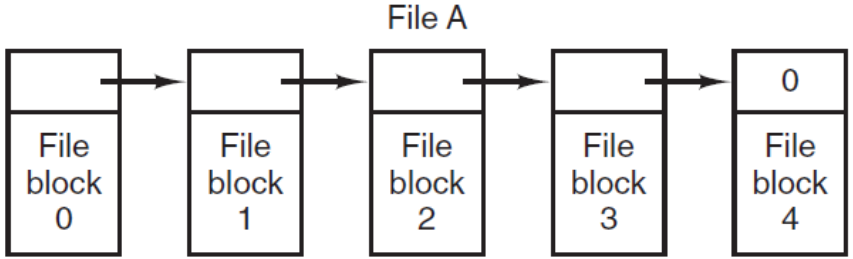


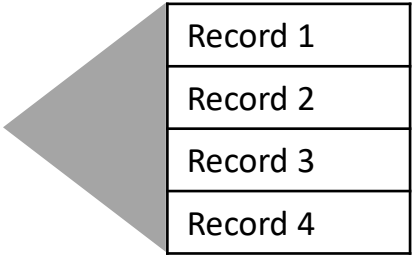


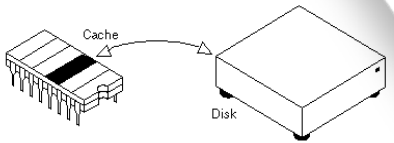


Agrupando Registros en Páginas

- **La paginación** se utiliza para la gestión de la memoria virtual, que puede incluir partes de archivos que se cargan en la RAM.
  - La paginación divide archivos grandes **en páginas más pequeñas de tamaño fijo**, no necesariamente del tamaño de un bloque.
  - La uniformidad del tamaño de las páginas simplifica la gestión de la memoria, ya que el sistema operativo puede realizar un seguimiento y asignar la memoria de manera más eficiente.







Agrupando Registros en Páginas

Se presentan dos casos:

1. La longitud del registro es ***menor*** que el tamaño del bloque
   - Este es el caso más común
   - El máximo número de registros que son almacenados dentro de un bloque es llamado **"blocking factor":** $b = \lfloor B/r \rfloor$, en donde:
     - **B** es el tamaño del bloque en bytes
     - **r** la longitud del registro en bytes

Agrupando Registros en Páginas

Se presentan dos casos:

2. La longitud del registro es ***mayor*** que el tamaño del bloque

- Se utiliza la organización extendida (spanned organization).
- Un registro extendido puede abarcar diferentes bloques.

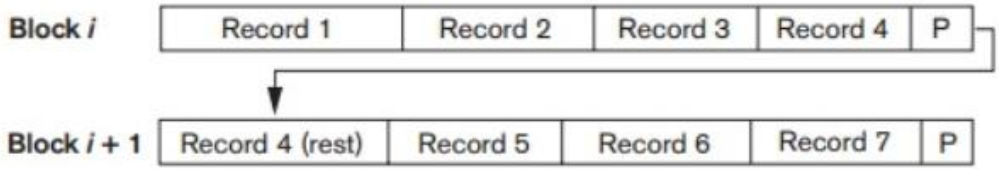
- File Blocks: secuencia de bloques que contienen todos los registros del archivo.

Agrupando Registros en Páginas

Se presentan dos casos:

2. La longitud del registro es ***mayor*** que el tamaño del bloque

- También se utiliza organización extendida para evitar espacios libres en los buffers.



Leer más …
File Structures: an object oriented approach with C++. Michael Folk. ***Chapter 4.***

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

> Reinventa el mundo <

4. Métodos de Organización de Archivos

B+Tree File

File Structures: An object-oriented approach with C++, Michael J. Folk. Addison Wesley, 3rd Edition, 1998.

Índice Dense vs Índice Sparse

Dense Index-file:

- Contiene una entrada para cada registro en el archivo de datos.

- Permite búsquedas muy rápidas de registros individuales, ya que cada registro tiene su propia entrada en el índice.

¿Ventajas y Desventajas?

Index.dat

Nombre	Ubicación
Carlos	R3
Fernando	R7
Jorge	R4
~~Keiko~~	~~R1~~
Marisol	R5
Rafael	R2
Ricardo	R6

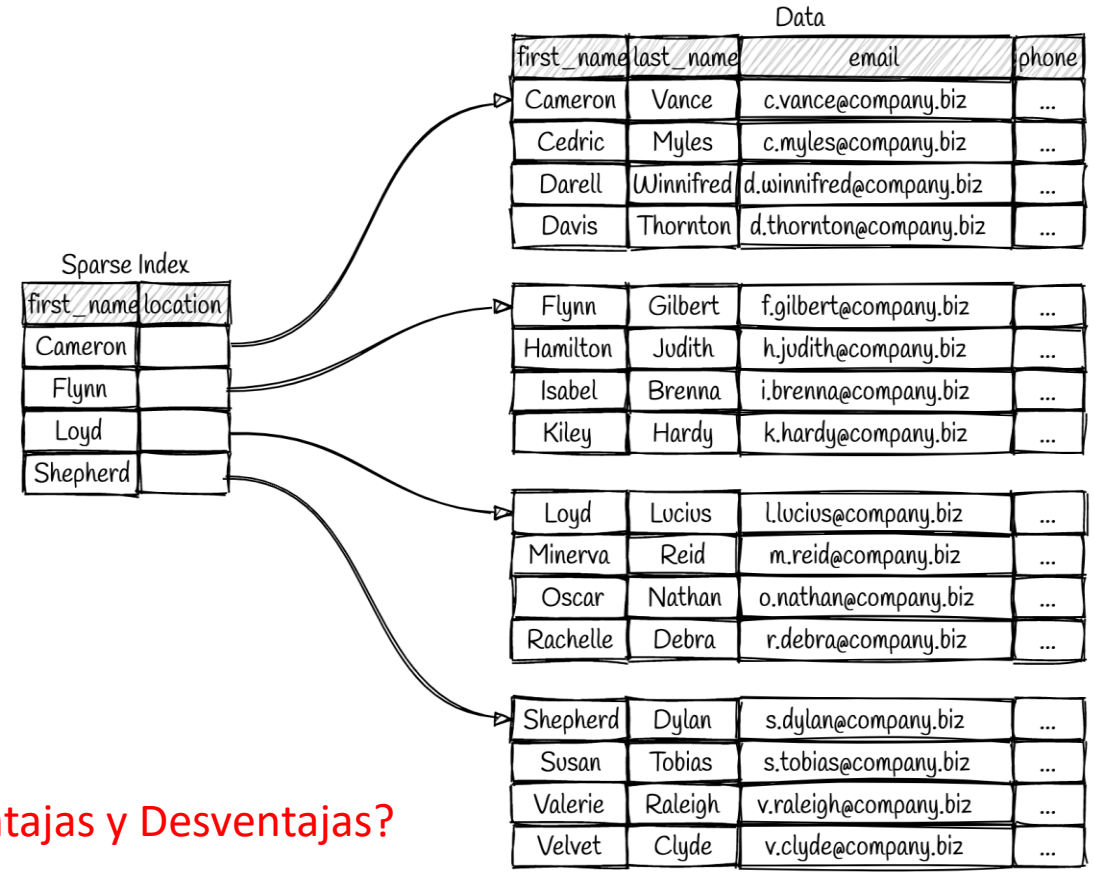
Data.dat

	Código	Nombre	Ciclo
~~R1~~	~~P-087~~	~~Keiko~~	~~5~~
R2	P-102	Rafael	7
R3	P-231	Carlos	3
R4	P-250	Jorge	5
R5	P-312	Marisol	1
R6	P-362	Ricardo	3
R7	P-982	Fernando	9

Índice Dense vs Índice Sparse

Sparse Index-file:

- Contiene entradas solo para algunos registros en el archivo de datos, generalmente el primer registro en cada página de datos.
- Para encontrar un registro específico, primero se busca en el índice el bloque que podría contener el registro y luego se busca secuencialmente dentro del bloque asociado.
- *Sólo posible si datos están ordenados.*

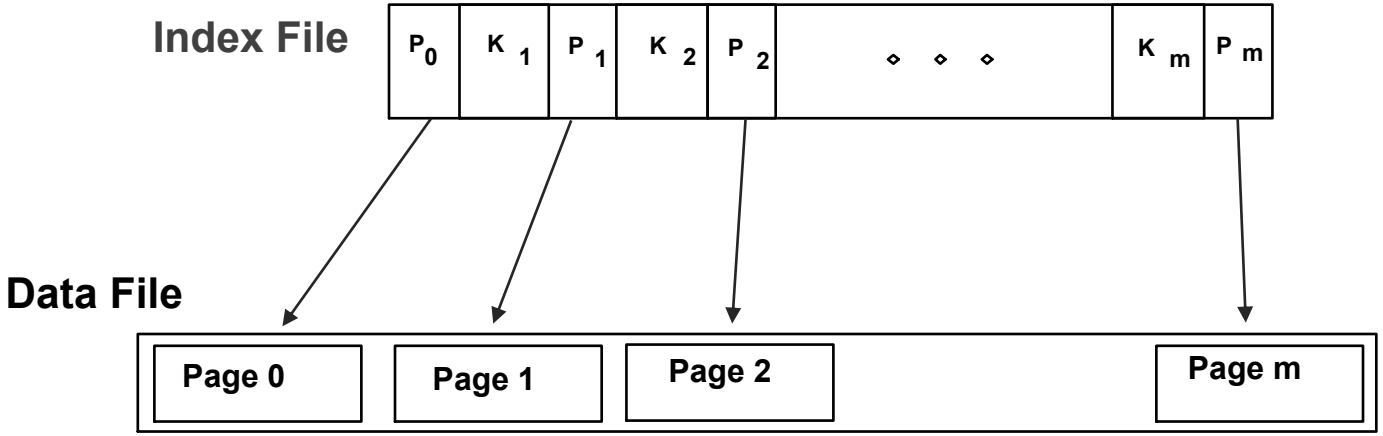


¿Ventajas y Desventajas?

Índice Dense vs Índice Sparse

Sparse Index-file:

- Construcción:

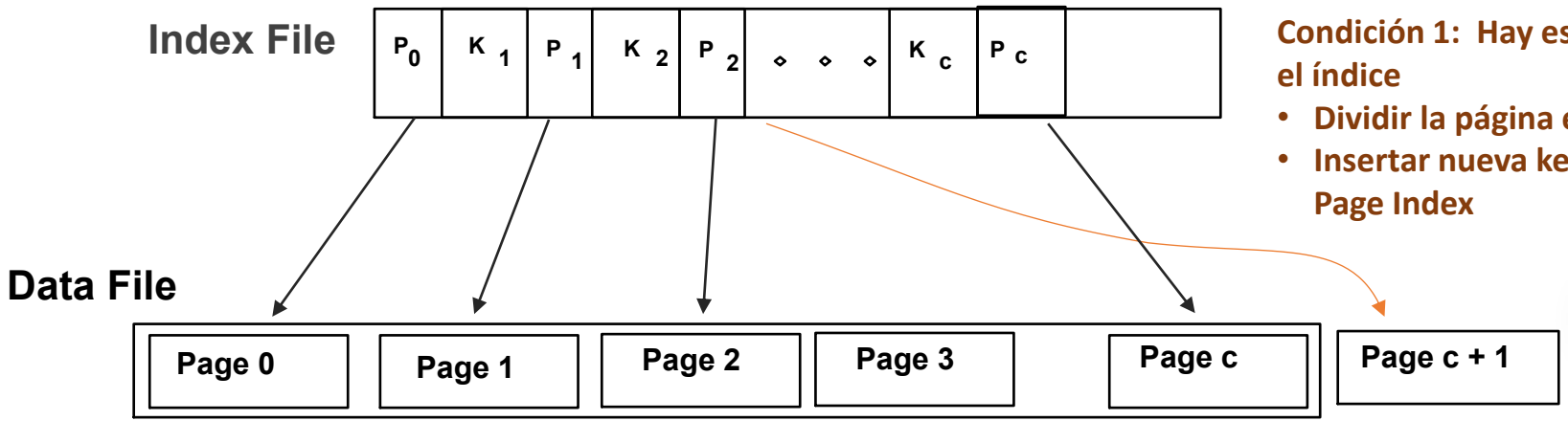


https://dsf.berkeley.edu/dbcourse/lecs/lecture5.pdf

Índice Dense vs Índice Sparse

Sparse Index-file:

¿Cómo insertar un nuevo registro?



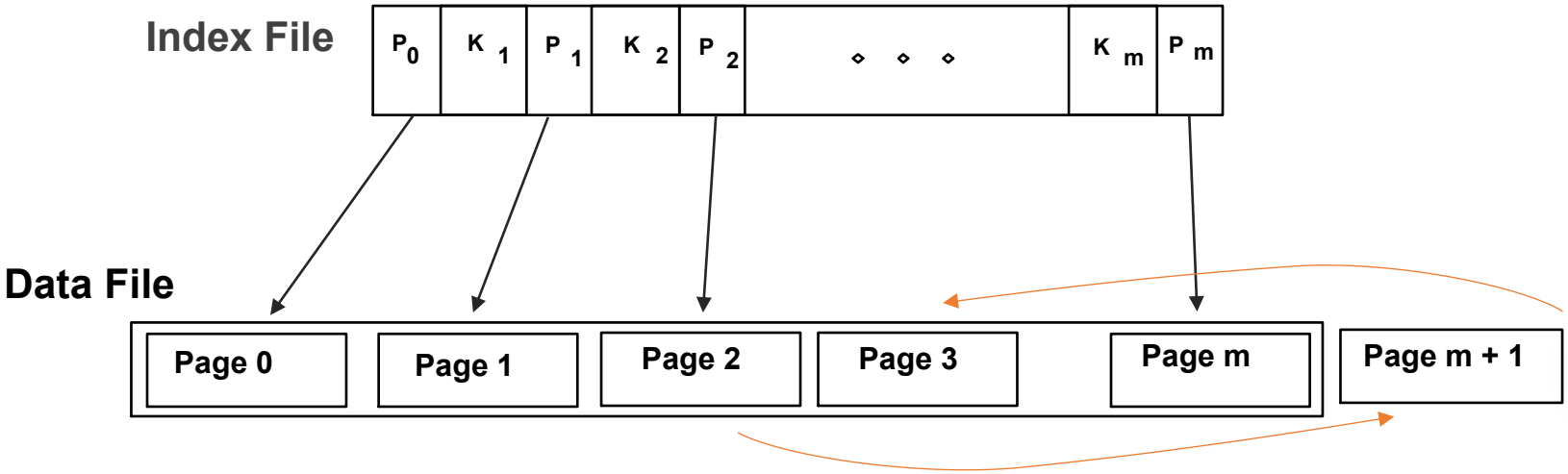
Condición 1: Hay espacio en el índice

- **Dividir la página en dos**
- **Insertar nueva key en el Page Index**

Índice Dense vs Índice Sparse

Sparse Index-file:

¿Cómo insertar un nuevo registro?



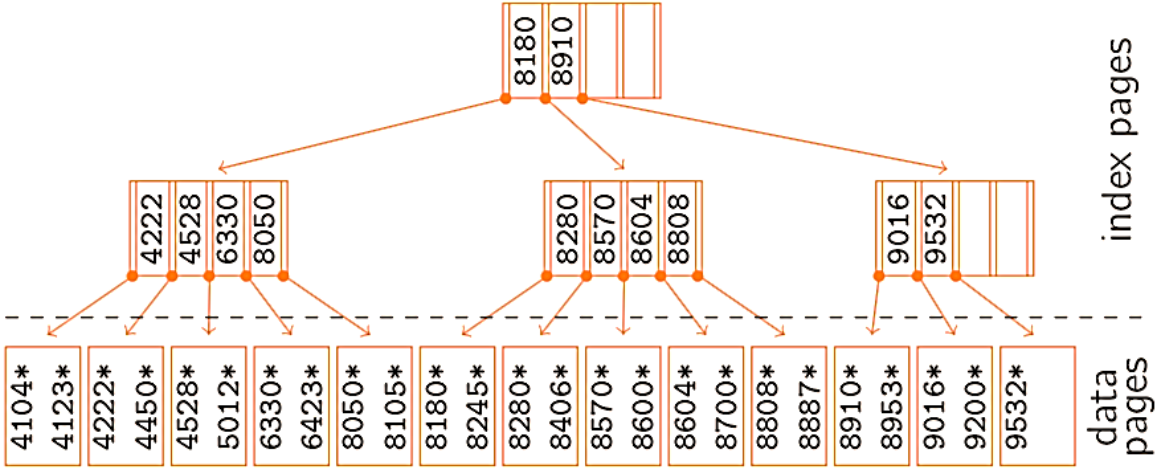
Condición 2: No hay espacio en el índice

- **Crear nueva pagina con un solo registro**
- **Encadenar nueva pagina a la pagina actual**

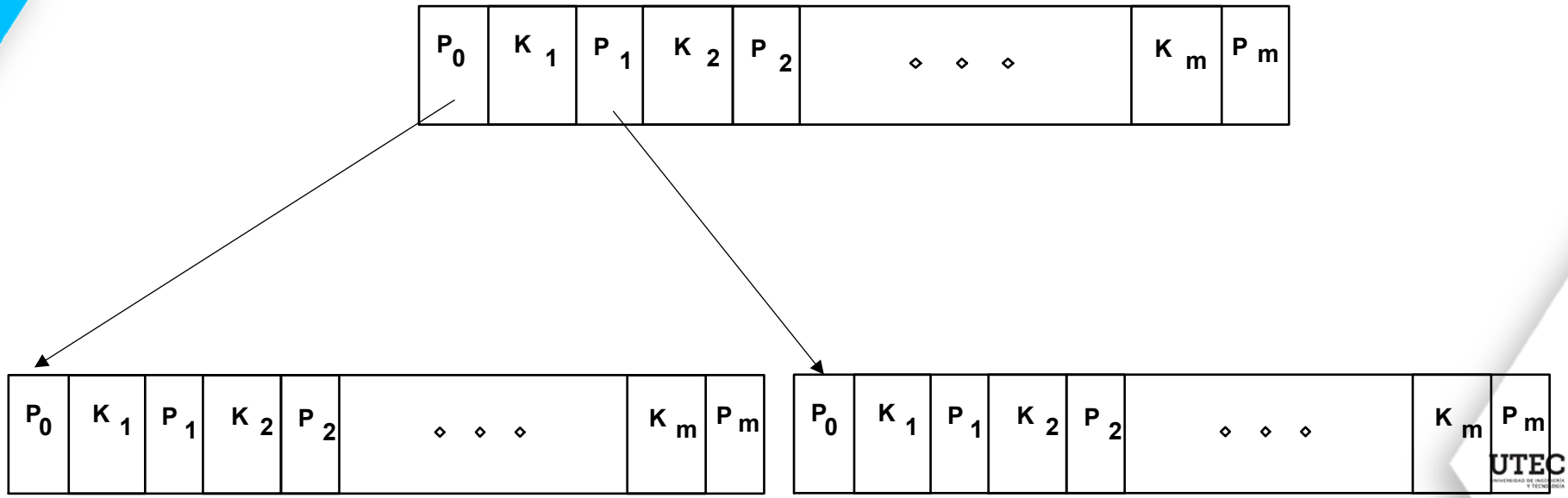
Índice Dense vs Índice Sparse

Multilevel Index-File:

- Cuando los archivos de datos son muy grandes, un índice de un solo nivel puede ser ineficiente para las búsquedas.
- La indexación multinivel soluciona esto al establecer una jerarquía de índices, donde el índice superior dirige a bloques de entradas en niveles inferiores, hasta llegar a los registros de datos.



Índice Dense vs Índice Sparse

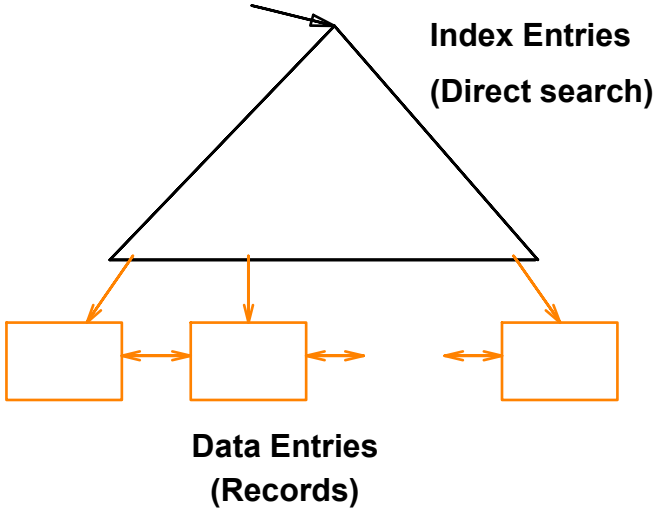


📌 El B+Tree combina ambos mundos: nodos hoja actúan como índice denso sobre las páginas de datos.

B+ Tree File

- Es una estructura de datos de árbol auto-balanceada que mantiene los datos ordenados y permite búsquedas, accesos secuenciales, inserciones y eliminaciones en tiempo logarítmico.
- Están optimizados para operaciones de E/S de disco, ya que minimizan el número de accesos al disco necesarios para encontrar un registro.
- Esto se logra al agrupar múltiples claves en cada nodo, lo que reduce la altura del árbol.

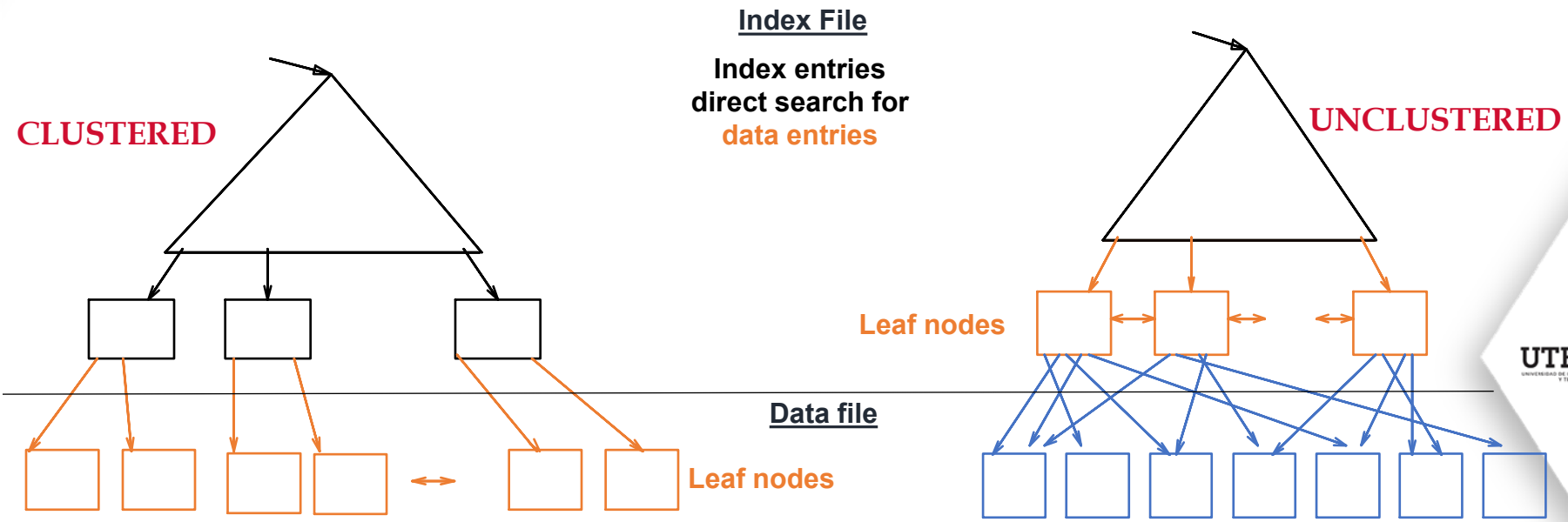
B+ Tree Index



Propiedades del B+ Tree:

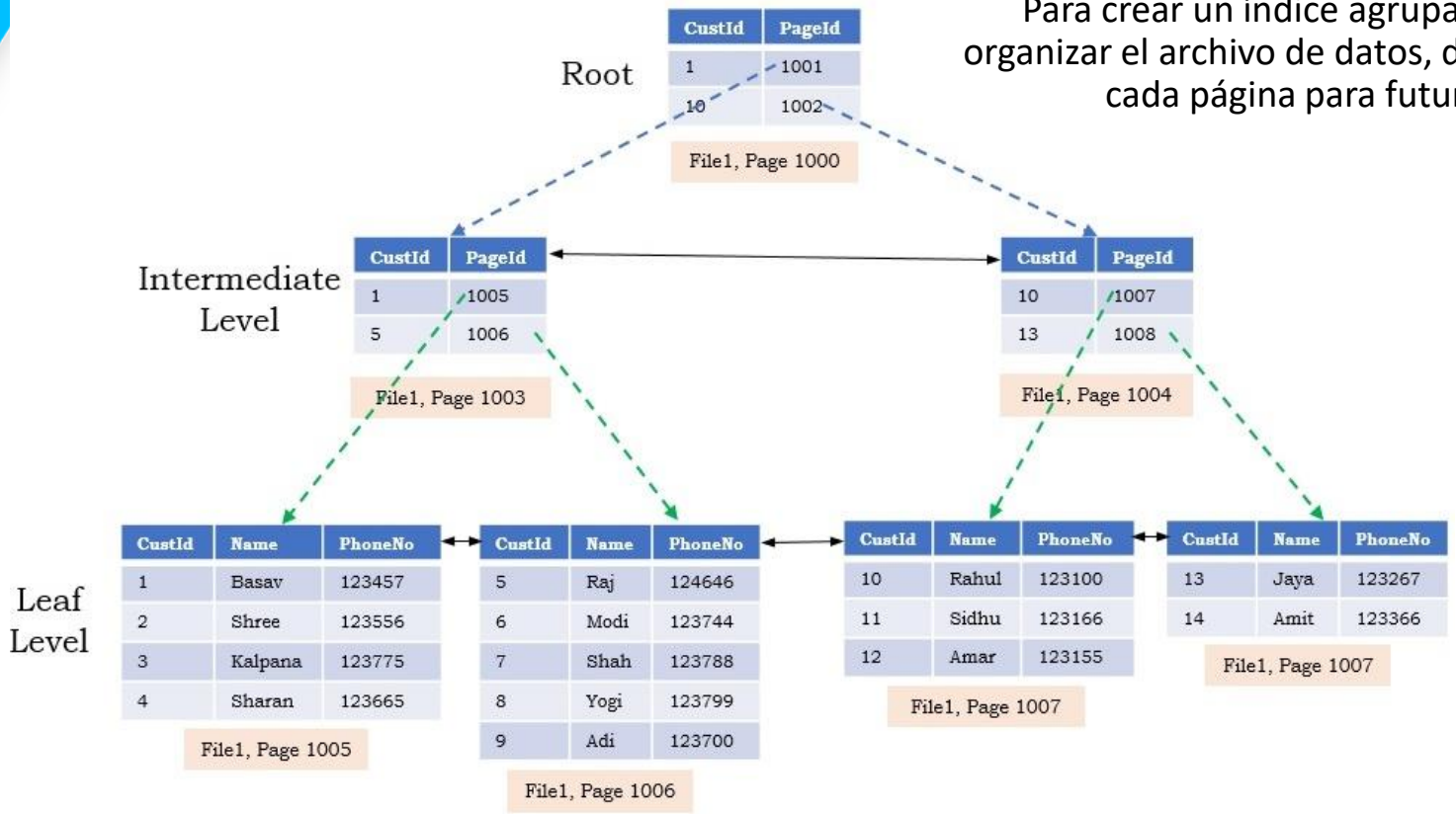
- Todos los nodos hoja deben estar al mismo nivel de altura.
- Los nodos hoja apuntan a los registros reales en el archivo de datos.
  - **Todos los nodos hoja están conectados de manera similar a una lista enlazada.**
- Los nodos internos forman el directorio del índice.
  - **Los nodos internos no contienen información del archivo de datos.**
- Los nodos deben estar siempre llenos al menos en su mitad.

B+ Tree Index: Clustered and Unclustered

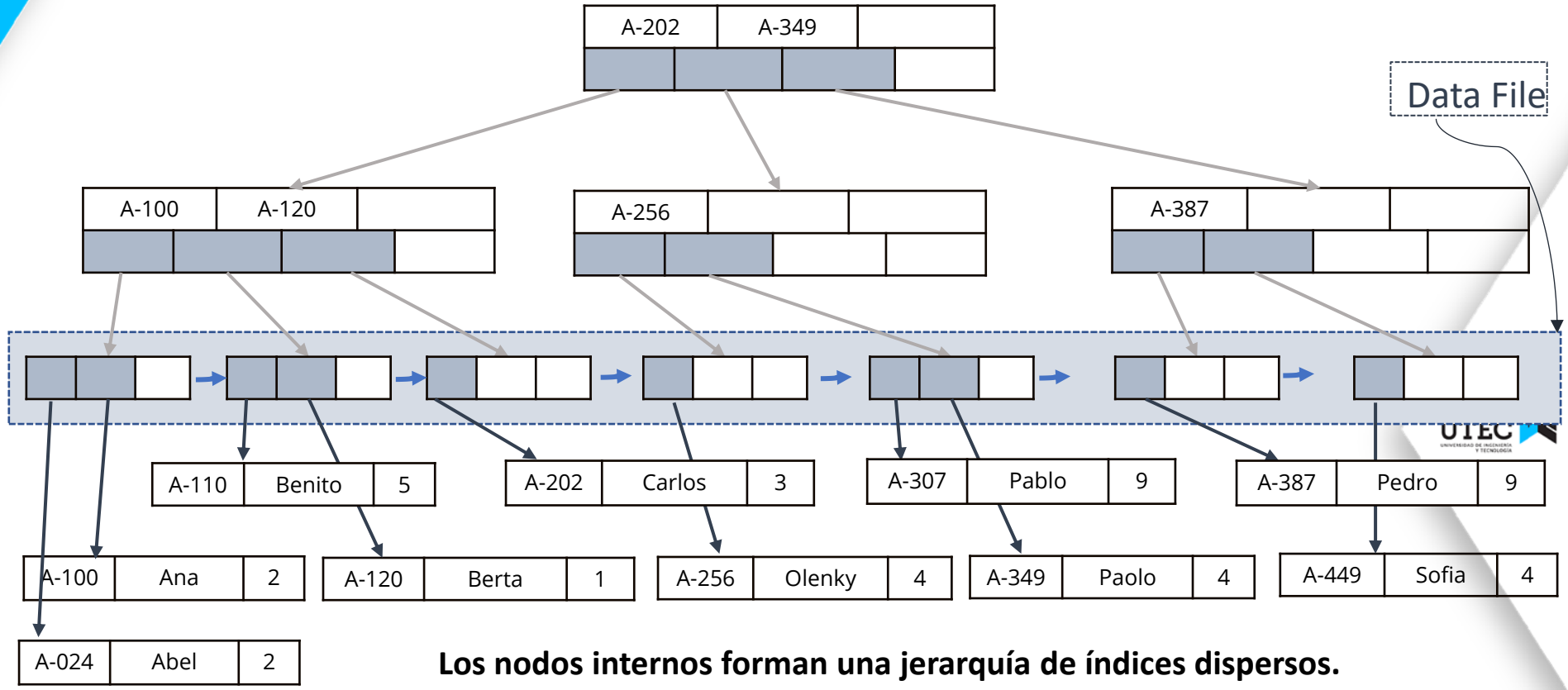


B+ Tree Index: clustered index

Para crear un índice agrupado, primero se debe organizar el archivo de datos, dejando espacio extra en cada página para futuras inserciones.



B+ Tree Index: clustered index



Los nodos internos forman una jerarquía de índices dispersos.

B+ Tree Index: clustered index

Características:

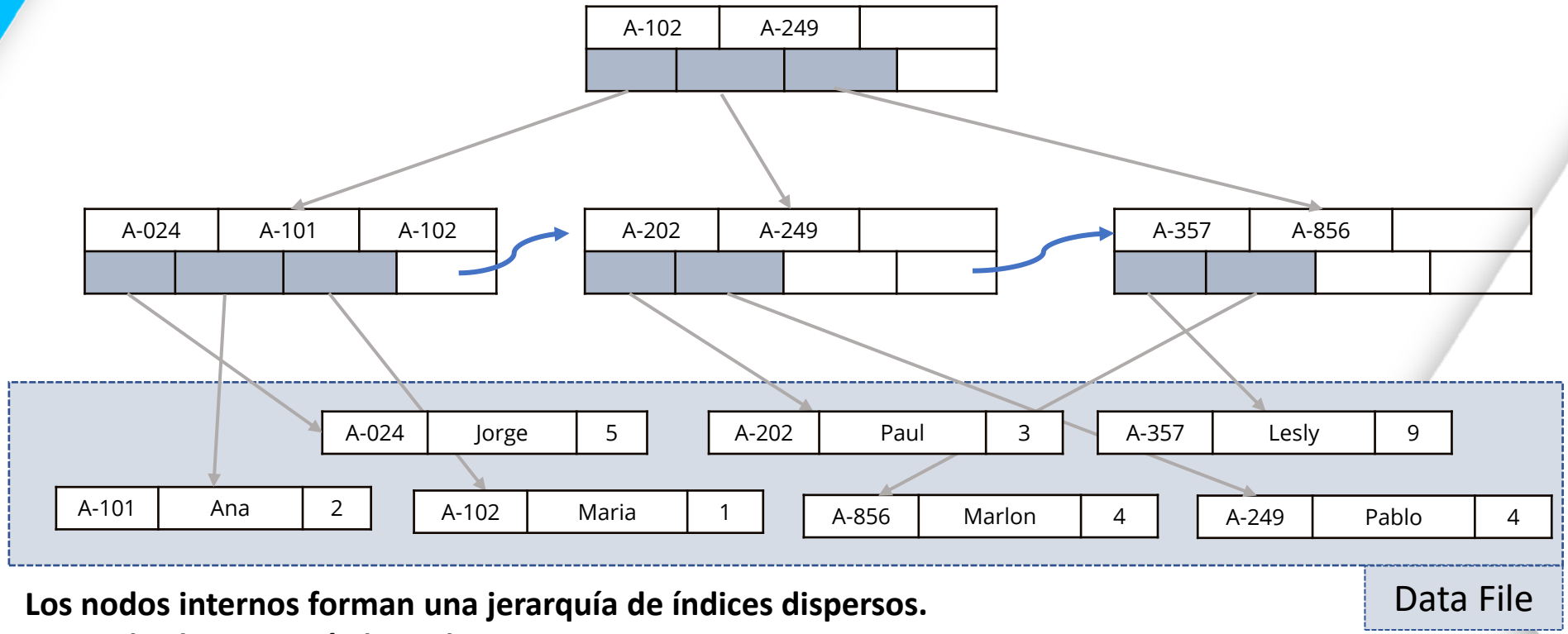
- Solo puede existir UN índice agrupado por tabla (el archivo tiene un único orden físico).
- Ventaja en Búsqueda por Rango [a, b]:
  - Se leen páginas contiguas → mínimo I/O.
- **En InnoDB (MySQL):**
  - La PK es siempre el clustered index; las hojas contienen el registro completo (índice en árbol = tabla).
- **En PostgreSQL:**
  - Se ejecuta CLUSTER table USING index — reorganiza el data file (heap), pero el orden no se mantiene automáticamente.

```
Índice B+Tree (clustered)          Archivo de datos (ordenado por k)

┌──────────────────────┐          ┌──────────────────────────────┐
│ Nodo hoja:           │          │ Pág 1: k=5  k=10  k=15  k=20 │
│ [5  → Pág1] ─────────┼──────────►                              │
│ [25 → Pág2] ─────────┼──────────► Pág 2: k=25 k=30 k=37  k=42  │
│ [48 → Pág3] ─────────┼──────────►                              │
└──────────────────────┘          │ Pág 3: k=48 k=52 k=60  k=75  │
                                  └──────────────────────────────┘

Orden del índice = orden físico de las páginas de datos
```

B+ Tree Index: unclustered index



Los nodos internos forman una jerarquía de índices dispersos.
Los nodos hojas son índices densos.

B+ Tree Index: unclustered index

Características:

- **Costo de range scan:**
  - Potencialmente O(1) I/O por registro si las páginas están dispersas → heap fetch.
- **Index-only scan:**
  - Si todos los campos de la consulta están en el índice, no se accede al heap.
- **Covering index:**
  - índice no agrupado que incluye todas las columnas necesarias → evita heap fetch.
- **PostgreSQL usa bitmap index scan para agrupar RIDs antes de leer el heap.**

```
Índice B+Tree (non-clustered)    Heap (datos desordenados)

┌──────────────────────┐        ┌──────────────────────────┐
│ Hoja: edad=22 → ─────┼────────┼──▶Pág 7: RID=(7, 3)     │
│       edad=25 → ─────┼────────┼──▶Pág 2: RID=(2, 1)     │
│       edad=30 → ─────┼────────┼──▶Pág 4: RID=(4, 2)     │
│       edad=35 → ─────┼────────┼──▶Pág 1: RID=(1, 4)     │
└──────────────────────┘        └──────────────────────────┘

RID = (número de página, slot)   →   saltos aleatorios a páginas
```

B+ Tree Index: Clustered vs. Unclustered

- Decimos que el índice está agrupado si está construido sobre el mismo atributo utilizado para ordenar los registros en el archivo de datos.
  - De lo contrario, decimos que es un índice no agrupado.
- Un archivo de datos puede agruparse como máximo en una clave de búsqueda.
  - Claramente, podemos construir como máximo un índice agrupado para cada tabla, sin embargo podemos construir un número arbitrario de índices no agrupados.

B+ Tree Index: Clustered vs. Unclustered

- **Índice Agrupado**
- Un solo índice clustered por tabla.
- Datos ordenados físicamente por la clave.
- Búsqueda puntual: O(log d N).
- Rango: O(log d N + páginas del rango).
- Ideal para: PK, búsquedas de rango.
- Inserción: puede causar page split en heap.

- **Índice No Agrupado**
- Múltiples índices por tabla.
- Estructura separada; apunta al heap.
- Búsqueda puntual: O(log d N) + heap fetch.
- Rango: puede ser costoso (random I/O).
- Ideal para: predicados de alta selectividad.
- Covering index elimina heap fetch.

B+ Tree Index

- **<u>Inserción (pasos generales):</u>**
  - Se recorre el árbol desde la raíz en busca del nodo hoja que va a contener al registro a insertar usando la clave de búsqueda.
  - Si la clave ya esta presente en el nodo hoja, agregar el registro al archivo de datos (agregar un puntero al bucket si es necesario).
  - Si la clave no esta presente, agregar el registro al archivo principal (crear un bucket si es necesario).
  - Si hay espacio en el nodo hoja, agregar el par (clave-dirección) en dicho nodo.
  - En otro caso dividir el nodo y actualizamos el nodo padre (actualización en cascada).

B+ Tree Index

- **<u>Eliminación (pasos generales):</u>**
  - Se recorre el árbol en búsqueda del nodo hoja que contiene el registro a eliminar.
  - Remover el par (clave-puntero) del nodo hoja.
  - Si el nodo se queda con pocas entradas debido a la eliminación realizada, realizar:
    - Si las entradas en el nodo y un hermano encajan en un solo nodo, fusione los hermanos.
    - Si no encajan, redistribuir los registros entre el nodo y un hermano de manera que ambos tengan más del número mínimo de entradas.
    - Actualice la clave de búsqueda el correspondiente en el nodo padre.
  - Las eliminaciones pueden ocasionar una actualización en cascada hacia arriba hasta que se encuentre un nodo con mas de la mitad de entradas.

B+ Tree Index

- **Complejidad:**
  - Cada nodo interno tiene entre $\left\lceil \frac{R}{2} \right\rceil$ y R entradas. Siendo R es el factor de bloque.
  - Cada nodo hoja tiene entre $\left\lceil \frac{R}{2} \right\rceil$ y $R - 1$ entradas. Ya que el ultimo espacio se reserva para el puntero al siguiente nodo.
  - Si consideramos M como el total de claves de búsqueda para construir el índice (unclustered), la altura del árbol no es mas que $\left\lceil log_{\lceil R/2 \rceil} M \right\rceil$. Este sería el costo de búsqueda.
  - Para inserciones y eliminaciones: costo de búsqueda + costo del split * nodos desbordados.

B+ Tree Index

- **Ventajas:**
  - Se reorganiza automáticamente con pequeños cambios locales cuando ocurren inserciones y eliminaciones.
  - No se requiere la reorganización de todo el archivo para mantener el rendimiento.
  - Soporta múltiples operadores de búsqueda de base de datos (order by, between, etc.)
- **Deficiencias:**
  - El rendimiento se degrada a medida que el archivo crece: demasiados bloques de desbordamiento.
- **Las ventajas del B+ Tree superan sus desventajas.**

Costo de operaciones (acceso a memoria secundaria)

	Scan	Equality	Range	Insert	Delete
Heap	MD	MD	MD	1D	MD
Sorted	MD	$Dlog_2(M)$	$Dlog_2\mathrm{M} + \#\mathrm{matches}$	1) $Dlog_2\mathrm{M} + cD$ 2) MD	1)$Dlog_2\mathrm{M} + c\mathrm{D}$ 2) MD
Static Hash	MD	$1D + collision$	MD	$D + collision$	$D + collision$
B+ Tree	MD	$Dlog_{R/2}(M)$	$Dlog_{R/2}(M)$ $+\#\mathrm{matches}$	$Dlog_{R/2}(M)$ $+reasignacion$	$Dlog_{R/2}(M)$ $+reasignacion$

M: Número de páginas/buckets **R:** Número de registros por página **D:** (Promedio) Costo de lectura/escritura en disco

Bitmap Index Scan en PostgreSQL : ¿Cómo funciona?

Problema: un índice non-clustered sobre `edad=25` retorna muchos RIDs dispersos → cada RID puede implicar un acceso a una página diferente del heap. **Solución PostgreSQL:** construir un bitmap de páginas en memoria antes de leer el heap.

① B+Tree Index Scan

(non-clustered, sobre edad)

Hoja B+Tree (edad=25):

- edad=25 → (pág 7, slot 2)
- edad=25 → (pág 2, slot 0)
- edad=25 → (pág 5, slot 3)
- edad=25 → (pág 1, slot 1)
- edad=25 → (pág 6, slot 0)

RIDs →

② Bitmap en Memoria

(1 bit por página del heap)

Bitmap (1 bit/página):

- 0 Pág 0
- 1 Pág 1 ← contiene edad=25
- 1 Pág 2 ← contiene edad=25
- 0 Pág 3
- 0 Pág 4
- 1 Pág 5 ← contiene edad=25
- 1 Pág 6 ← contiene edad=25
- 1 Pág 7 ← contiene edad=25

páginas →

③ Heap Fetch (ordenado)

(páginas en orden físico)

Heap leído en orden físico:

- **Pág 1** → leer 1 vez *(buscar slots con edad=25)*
- **Pág 2** → leer 1 vez *(buscar slots con edad=25)*
- **Pág 5** → leer 1 vez *(buscar slots con edad=25)*
- **Pág 6** → leer 1 vez *(buscar slots con edad=25)*
- **Pág 7** → leer 1 vez *(buscar slots con edad=25)*

📌 Beneficio: los 5 RIDs dispersos se convierten en 5 accesos secuenciales a páginas ordenadas → se evita el problema de thrashing de caché en heap random-access.

Bitmap Index Scan — Combinación de índices con AND / OR

Query: `SELECT * FROM empleados WHERE edad = 25 AND depto = 'TI'`

PostgreSQL construye un bitmap por cada índice y luego hace AND bit a bit antes de acceder al heap:

Bitmap A *idx_edad (edad=25)*	Bitmap B *idx_depto (depto='TI')*	AND *Intersección de páginas*	OR *Unión de páginas*
0 Pág 0	0 Pág 0	0 Pág 0	0 Pág 0
1 Pág 1	1 Pág 1	1 Pág 1	1 Pág 1
0 Pág 2	1 Pág 2	0 Pág 2	1 Pág 2
0 Pág 3	0 Pág 3	0 Pág 3	0 Pág 3
1 Pág 4	0 Pág 4	0 Pág 4	1 Pág 4
1 Pág 5	1 Pág 5	1 Pág 5	1 Pág 5
0 Pág 6	1 Pág 6	0 Pág 6	1 Pág 6
1 Pág 7	0 Pág 7	0 Pág 7	1 Pág 7

AND → leer solo págs: `1, 5` **OR → leer págs:** `1, 2, 4, 5, 6, 7` → *siempre en orden físico, 1 sola una pasada por el heap.*

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA